CS6290 HPCA — Midterm Cheat Sheet

Intro · Metrics · Pipelining · Branches · Predication · ILP · Tomasulo · ROB · Memory Ordering · Compiler ILP · VLIW · Caches · VM · Advanced Caches

Formulas

Power	$P_{dyn}=½CV^2f\alpha$; lower V → leakage↑ → optimal V exists
Cost	wafer cost / #good chips; bigger die → super-linear cost (fewer dies + lower yield)
Speedup	$Lat_{old}/Lat_{new} = Thr_{new}/Thr_{old}$
Iron Law	$IC \times CPI \times T_{clk}$; mixed: $\Sigma\ IC_i CPI_i \times T_{clk}$
Amdahl	`1/[(1−F)+F/S]`, **F = fraction of TIME**
Means	avg times then ratio, or geo-mean both; never arith-mean speedups
Pipeline	n instr, k stages: k+n−1; `CPI = 1 + Σ freq×cost`
Branch	`CPI + mispred/branch × branch/inst × penalty`; resolve in stage k → penalty k−1
History	N-bit: $N + 2\cdot2^N$ bits/entry; predicts patterns length ≤ N+1
AMAT	$HT + MR\times MP$; $MP_{L1} = HT_{L2} + MR_{L2,local}\times MP_{L2}$
Cache	offset=$\log_2$B; #sets=size/(B×ways); index=$\log_2$sets; tag=rest
Flat PT	`(VA/page) × PTE` — independent of usage/phys. mem
VIPT	no aliasing iff `size ≤ ways × page`
Prefetch	`pdist ≈ mem latency / cycles per iter`; check not evicted

1 · Intro

- Arch = design for **intended use** (perf, battery, size, weight, cost, new abilities). Faster transistors ≠ arch.
- Moore: 2× transistors / 18–24 mo → 2× speed, ½ energy/op, 2× memory. **Memory wall**: latency only 1.1×/2yr → caches.
- Same tech step: 2× speed same cost/power, OR 1.1× speed ½ cost ½ power.

Ex. 30W @1V/2GHz → 0.9V/1.8GHz: 30·0.81·0.9 = 21.9W; 1.5V/3GHz: 30·2.25·1.5 = 101.3W.

Ex. Wafer $5000, 10 defects: 400→390 good $12.8; 96→86 $58.2 (~5×); 20→11 $454. 100mm² vs 1mm² → cost >100×.

2 · Metrics

- Latency ≠ 1/throughput (2 servers × 1ms → 1ms, 2000/s). 4h→10min: speedup 24 (reverse 0.04).
- Benchmarks: real apps → **kernels** (part of real code) → **synthetic** (not real code; design studies) → peak (marketing). TPC (DB/web), EEMBC (embedded), SPEC (CPU).
- Iron Law: IC ← alg, compiler, ISA; CPI ← ISA, µarch; T ← µarch, circuits, tech.

Ex. 50B instr: 10B br×4 +15B ld×2 + 5B st×3 + 20B×1 = 105B cyc / 4GHz = 26.25s.

⚠ Amdahl F from **instruction** mix is wrong. int40%×1, br20%×4→2, ld30%×2, st10%×3: CPI 2.1→1.7 = **1.24** (not 1.11).

- Common case fast: 20× on 10% → 1.105; 1.6× on 80% → 1.43; ∞ on 10% ≤ 1.111. Lhadma: 2× on 90% but 10× slower on 10% → 0.69.
- Diminishing returns: 50/50, 2× → 1.33; again 2× → 1.2. Re-evaluate the dominant part.

3 · Pipelining

- F D A M W. Laundry 10×3 stages: 30 → 12h. 5 stages 10 instr: 50 → 14 cyc.
- CPI>1: stalls (data), flushes (control). 20% br, 50% taken, resolve stage 3 → 1+0.1×2 = 1.2. 25% taken, resolve stage 6 → 1+0.25×5 = 2.25.
- RAW = flow/true; WAR = anti, WAW = output (false/name); RAR none.
- Dependence = program; **hazard** = dependence that breaks in this pipeline. 5-stage: WAR/WAW never hazards; RAW hazard if <3 instr apart. 3-stage: adjacent only.
- Control → **flush**; data → **forward** (prefer) or **stall**; load-use: stall 1 then forward.
- Depth: perf-optimal 30–40; with power → 10–15 stages.

4 · Branch Prediction

- At fetch only PC known; need "taken branch?" + target. No prediction: non-branch 2 cyc, branch 3. Not-taken: 1 / 1 or 3 → always ≥ no-predict.
- Mispredicts in the shadow of another add **no** penalty.

Ex. Not-taken ≈88% (12% mispred): 5-stg 1.24; 14-stg(pen 10) 2.2; 4-wide 0.25+1.2=1.45. 99%: 1.02/1.1/0.35. No-predict vs perfect 3-instr loop 7 vs 3. P4: 0.5 = x+0.2·0.01·30 → x=0.44; 2% → 0.56.

- **BTB**: PC → target; small, 1-cycle. Index = lowest **varying** PC bits (4B instr: drop 2). 1024 entries, PC 0xAB0C → bits[11:2] = 0x2C3.
- **BHT**: tiny entries, accessed every fetch; BTB only if predicted taken (perfect BHT → #taken).
- 1-bit: 2 mispredicts per anomaly; bad on short loops.
- **2BC**: 00 SN, 01 WN, 10 WT, 11 ST; MSB predicts; saturating ±1. Anomaly = 1 miss; switch = 2. Init state barely matters. Every predictor has a 100%-miss sequence (00: T,T,N,T,N…). 3-bit only for streaky anomalies.
- **History**: N-bit local hist + 2^N 2BCs. 1-bit hist: NT ok, NNT → 1 miss/round. 2-bit hist: NNT ok (3 of 4 ctrs). Only ~N+1 ctrs used. 8-iter loop → 8-bit hist.

N	bits/entry	×1024	NNNT	NT ctrs
1	5	5,120	✗	2/2
4	36	36,864	✓	2/16
8	520	532,480	✓	2/256
16	131,088	134,234,112	✓	2/65536

- **PShare**: PC→PHT (private history) XOR PC → shared 2BC table. $2^{11}\times11+2^{11}\times2$ ≈ 26Kbit. Good: self-patterns (even/odd, short loops).
- **GShare**: global history XOR PC; good for **correlated** branches; needs longer history (even/odd in loop: PShare 1 bit, GShare 3).
- **Tournament**: 2 good predictors + meta 2BC/PC; meta changes only when one right, one wrong.
- **Hierarchical**: cheap + expensive; expensive (tagged) only gets branches cheap one misses. Pentium M: 2BC → local → global. Template: **Hier(2BC, Tournament(PShare, GShare))**.
- **RAS**: push on call, pop on ret. Full → **wrap around** > don't push. Know it's RET via 1-bit predictor or **predecoding** (extra bits in I-cache).

5 · Predication

- Predict: loops, call/ret, large if-else. Predicate: small if-else w/ poor accuracy (5+5 instr: waste 5 vs 0.1×50).
- `MOVZ Rd,Rs,Rt`: Rt==0 → Rd←Rs; `MOVN`: Rt≠0. x86 CMOVcc.
- `BEQZ R1,Else; R2++ | Else: R3++` → `ADDI R4,R2,1; ADDI R5,R3,1; MOVN R2,R4,R1; MOVZ R3,R5,R1`.
- CMOV: needs compiler, extra regs, extra instr. **Full predication** (Itanium): 6-bit qualifying predicate in 41-bit instr, 64 1-bit pred regs; `cmp.eq p1,p2=R1,0; (p2)R2++; (p1)R3++`.

▶ Break-even: converted cost = branchy avg + p×penalty. 4 vs 2.5+30p → p=5% (if-convert if acc <95%). Full-pred 1.5 vs 1.25+10p → acc <97.5%.

6 · ILP

- Renaming removes WAR/WAW. RAT: arch → phys reg.

▶ Rename: read **sources** from RAT first, then new phys reg for **dest**.

ILP = IPC of ideal CPU (1-cycle instr, infinite width, perfect same-cycle branch pred, no structural limits) — **program property**. Only RAW edges; cycle = 1+max(producers); ILP = #instr / cycles. IPC ≤ ILP.

- IPC also limited by width, in-order, #units (ILP 2.5; 1 ALU → 1.25). Narrow in-order: width-limited; wide in-order: order-limited → need wide + OoO + renaming.

7 · Tomasulo

- IQ → per-unit RS; RAT; RF; CDB broadcast; L/S in order (original).
- **Issue** (in order): free RS of right type else stall; RAT empty → RF value, else RS tag; RAT[dest] ← RS#.
- **Dispatch** (OoO): all operands ready; 1/unit/cycle; oldest-first typical.
- **Broadcast**: (tag,val) on CDB → RSs capture; if RAT[reg]==tag → write RF, clear RAT; free RS. 1 CDB conflict: slower unit first.

⚠ **Stale broadcast** (tag no longer in RAT): RSs capture only; **no RF write, no RAT change**.

Defaults (check problem!): no issue→dispatch same cycle; no capture→dispatch same cycle; RAT written by issue & broadcast same cycle → issue wins. Freed RS reusable next cycle.

▶ Issue = prev+1 (or after RS frees); Exec = max(Issue+1, last operand bcast+1); Write = Exec+lat; delay on CDB conflict. Note reason for each delay.

8 · ROB

- Fixes: imprecise exceptions, mispredict recovery, phantom exceptions. Execute/broadcast OoO, **commit in order**.
- Entry: value, done, dest reg; issue ptr + commit ptr.

	Tomasulo	+ROB
Issue	RS; RAT→RS#	RS + ROB entry; RAT→ROB#
Dispatch	keep RS	**free RS**
Bcast	RF, RAT, free RS	tag=ROB#; RS capture; ROB val, done; RF/RAT untouched
Commit	—	head & done: **always** write RF; RAT→RF only if RAT==this ROB#; free

- ROB does: order, hold result, name/tag. Not: source operands, unit choice.
- Mispredict: recover when branch **commits**: empty ROB, RAT→RF, flush RS/units, refetch. Exception = result in ROB; handled at commit.
- Commit = max(Write+1, prev commit+1). Check if RS frees at dispatch or broadcast.
- Unified RS: better utilization, harder dispatch. Superscalar: every stage >1/cycle; narrowest limits.
- Terms: issue = allocate (= dispatch); dispatch = execute (= issue); commit = retire = graduate. In order: fetch, decode, issue, commit; OoO: dispatch, exec, bcast.

9 · Memory Ordering

- Stores write memory at **commit**. LSQ (in order): L/S, addr, value, done.
- Load: newest older store w/ same addr → **store-to-load forwarding**; else cache.
- Unknown older store addr: (1) all L/S in order; (2) wait for older store addrs; (3) **go anyway** + replay load & dependents if a store later matches (modern).
- Issue L/S: ROB + LSQ entry. Load broadcasts; store doesn't, no ROB result. Forwarded value comes from **LSQ**.

Ex. 3 LW/SW pairs, 40-cyc misses: OoO 44; in order 126.

10 · Compiler ILP

- Tree height reduction (needs associativity): 6-op chain ILP 1 → 2.
- Scheduling: fill stalls; moving ADDI R1 past SW → `SW -4(R1)`; may need renaming.
- If-conversion enables cross-path scheduling; loops → unrolling.

⚠ Unroll **once** = 2 iterations per loop; twice = 3. Adjust offsets, step 2×, one branch.

- Unrolling: IC↓ (5000→4000) + CPI↓ (3/5 → 2/5 sched; unrolled sched 3/8). Cons: code bloat, unknown trip count.
- Inlining: removes call/ret overhead + better scheduling (10→7 cyc); bloat → small functions only.
- Software pipelining; trace scheduling (+ compensation code).

11 · VLIW

	OoO SS	InO SS	VLIW
finds ILP	window ≫N	next N	compiler
HW cost	high	lower	lowest
compiler	helps	needs	depends fully

- 4000B program → 4000–16000B (NOPs). Good: regular loops, simple/efficient HW. Bad: variable latency, irregular code, bloat.
- ISA: full predication, many regs, branch hints, compaction (stop bits). 2×old-VLIW/cycle ≠ VLIW. Itanium bad; DSP good.

12 · Caches

- Temporal (j, sum) / spatial (arr[i]). L1 16–64KB, 1–3 cyc, ~90%. Block 32–128B, power of 2, aligned.
- Tag: no offset bits, no index bits. Valid=0 at boot; hit = tag match & valid.
- DM (fast, conflicts), N-way SA ("2-way" = 2 blocks/set), FA (no index).

▶ Offset first, then index, then tag. 16KB DM 256B: off 8, idx 6. 256B 32B 2-way 32b: off[4:0] idx[6:5] tag[31:7]. 4KB 4-way 64B 64b: 6/4/54; overhead V+D+tag+LRU = 58 bits. Conflict = same index, different block.

Ex. 32KB/64B, N vars no spatial locality → N ≤ 512.

▶ LRU: accessed → N−1; counters > old value −1; evict 0; must stay distinct.

- Write-allocate vs no; write-through vs **write-back** (dirty bit). WB + WA. 1-line DM: RA RB WB RC RD WD → 4 misses, 1 WB.

13 · Virtual Memory

- Pages ↔ frames, per-process PT, shared = same frame, rest on disk. PA = PFN | offset.
- 2GB phys / 4KB → 2^{19} frames; 4GB VA → 2^{20} PTEs. 4GB/4KB×4B = 4MB/process. 2 procs, 8B PTE → 16MB.
- Multi-level: omit inner tables for unused regions. 32b, 10|10|12, 8B, low+top used: 8MB flat → 8KB+2×8KB = **24KB**. 64b, 64KB pages, 4 levels ×12 bits, 4GB used: 16+1+1+1 = 19 × 32KB = **608KB** (flat 2^{51}B).
- Bigger pages: smaller PT, more internal fragmentation (x86 4KB).
- 3-level walk, mem 10, hit 1, 90%: uncached PTE 33 cyc; cached 9.
- **TLB**: final translations only; FA/high-assoc, 64–512; L2 TLB. SW miss (any PT format) vs HW walker (x86). 32KB/64B cache, 4KB page → 8–512 entries.
- 1MB ×10 sweeps, L1 TLB 128, L2 1024: L1 miss 10×256; L2 miss 256, hit 9×256.

14 · Advanced Caches

- **Hit time**: smaller/less assoc; pipelined cache; PIPT (TLB+cache serial); VIVT (perm check anyway, flush on ctx switch, **aliasing**); **VIPT** (parallel, no flush, no aliasing if index⊂page offset).

VIPT max = ways × page: 4-way, 16B, 8KB page → 32KB. Intel: 4-way 16KB (P4), 8-way 32KB, 16-way 64KB.

- **Way prediction** (not for DM): 0.7×1+0.3×2+0.1×20 = 3.3 vs 8-way 4, 4KB DM 7.
- LRU updates on every hit (slow). **NMRU**: $\log_2$N bits/set. **PLRU**: 1 bit/line, set on access, all 1 → clear others. NMRU quiz: 5–6 misses.
- **3Cs**: compulsory (first ref); capacity (misses even in FA same size); conflict (FA same size would hit).
- Larger blocks: help with spatial locality (↓compulsory, capacity, maybe conflict); turn-up later in big caches.
- Prefetch: pollution; too early/late. b: pdist 200/10 = 20; a: i+1 (8000B/outer <16KB). HW: stream buffer, stride, correlating. **Loop interchange**: row-major → j inner.
- **Miss penalty**: non-blocking, hit-under-miss, miss-under-miss, MLP. **MSHR**: miss → allocate; half-miss → append. 1 miss/1000 instr → no benefit.
- Multi-level: 2+0.1×(10+0.25×100) = **5.5**. Local vs global MR (L1 10%; L2 local 50% → global 5%). MPKI.
- **Inclusion**: not automatic (L2 LRU evicts L1-hot); inclusion bit. With inclusion, L1 WB always L2 hit.

Exam tip: copy the given rules (latencies, RS free time, same-cycle rules, #CDB) before filling Tomasulo/ROB/cache tables; annotate delays.